॥ वसुधैव कुटुम्बकम् ॥

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re-accredited by NAAC with "A" Grade

Name of the Student: Vatsal Postiwala **PRN: 26070122509**

Semester: III **Year: 2025-29**

Subject Title: Operating Systems Lab

EXPERIMENT No: 12

TITLE: LRU Page Replacement Policy **DoP**: 12/10/26

Aim: Implement a C program to demonstrate the Least Recently Used algorithm.

Learning Outcome: 1. To understand the Least Recently Used algorithm
2. To Demonstrate the Least Recently Used algorithm

Hardware/Software:

- Hardware requirements:
  - A PC with a minimum of 4GB RAM.
  - A processor with 1.6GHz clock speed or faster.
  - Sufficient hard disk space to store the code and compiler.
- Software Requirements:
  - Visual Studio Code (VS Code)
  - C++ compiler such as GCC
  - C/C++ extension for VS Code. This extension is used to compile and run C++ code directly from VS Code.

Theory:

Page replacement algorithms are used in operating systems to manage memory allocation for processes. Page replacement algorithms determine which page should be replaced in memory when a new page needs to be loaded and there is no free space available in memory. One such algorithm is the Least Recently Used (LRU) algorithm. The LRU algorithm works on the principle that the least recently used page is the most likely candidate for replacement, as it has been in memory for the longest time without being accessed and therefore, it should be replaced.

The LRU algorithm aims to minimize page faults by replacing the least recently used page in memory. It operates on the principle that pages that have been used recently are likely to be used again soon, while pages that have not been used recently are less likely to be used in the near future. The LRU algorithm keeps track of the time when each page was last accessed and replaces the page that was accessed the longest time ago when a new page needs to be loaded into memory.

The features of the LRU algorithm include:

- Minimization of page faults: The LRU algorithm aims to minimize page faults by replacing the least recently used page in memory.
- Simple to understand and implement: The LRU algorithm is easy to understand and implement in most systems.
- Effective utilization of memory: The LRU algorithm uses memory efficiently by replacing pages that are not likely to be used soon.

The drawbacks of the LRU algorithm include:

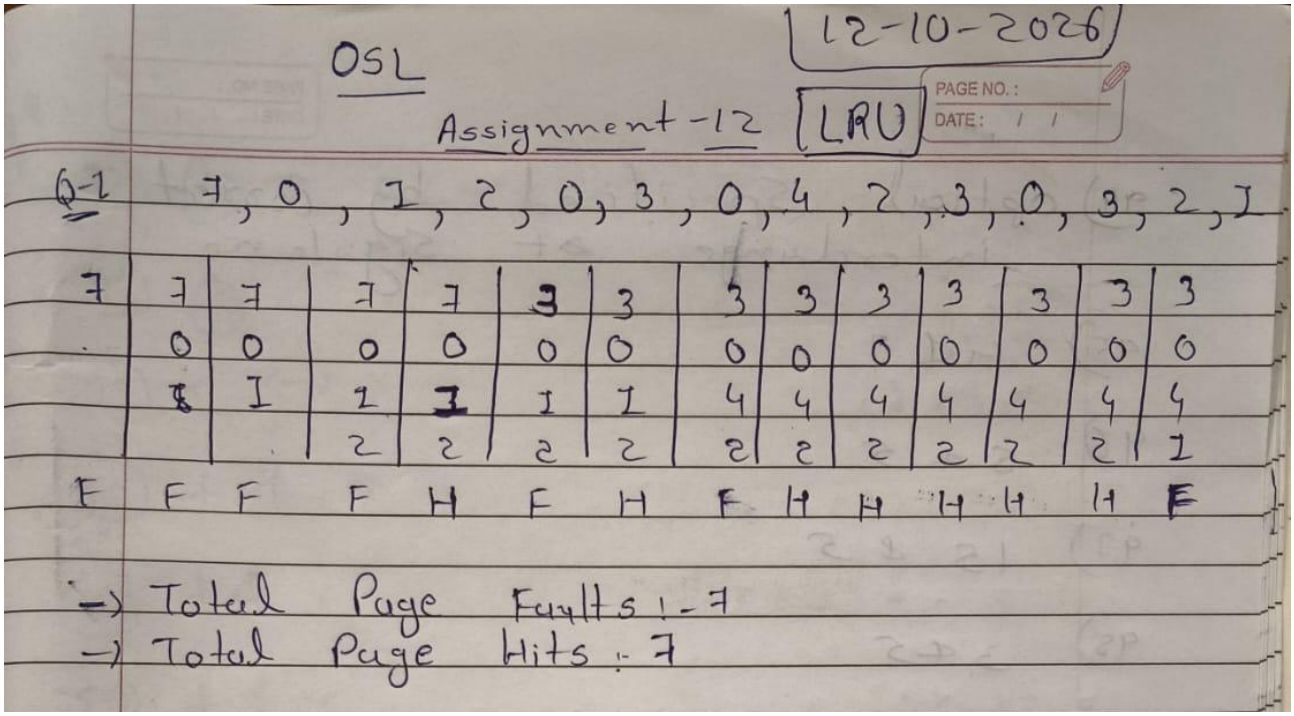
- Implementation overhead: The LRU algorithm requires additional overhead to keep track of the time when each page was last accessed, which can reduce system throughput.
- Not suitable for certain workloads: The LRU algorithm may not be suitable for workloads that access pages in a non-uniform or unpredictable manner, as it may lead to unnecessary page replacements and increased overhead.
- Possible thrashing: The LRU algorithm may suffer from thrashing, which occurs when the system spends more time swapping pages in and out of memory than executing processes.
- Not optimal in certain scenarios: The LRU algorithm may not be optimal in scenarios where certain pages need to be kept in memory for an extended period or where a subset of pages is accessed frequently.

Program Statements:

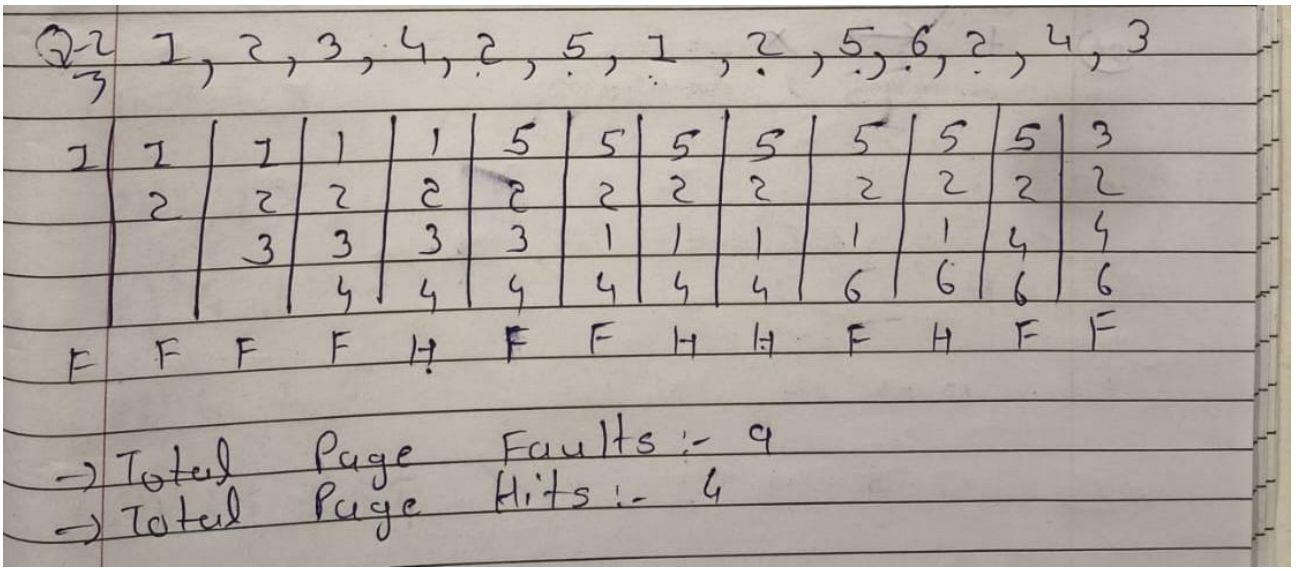
Consider a system with 4 page frames that uses the Least Recently Used (LRU) page replacement algorithm.

The following page reference string is given:

1. 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1



2. 1, 2, 3, 4, 2, 5, 1, 2, 5, 6, 2, 4, 3



Algorithm:

* Algorithm:-

1- Start

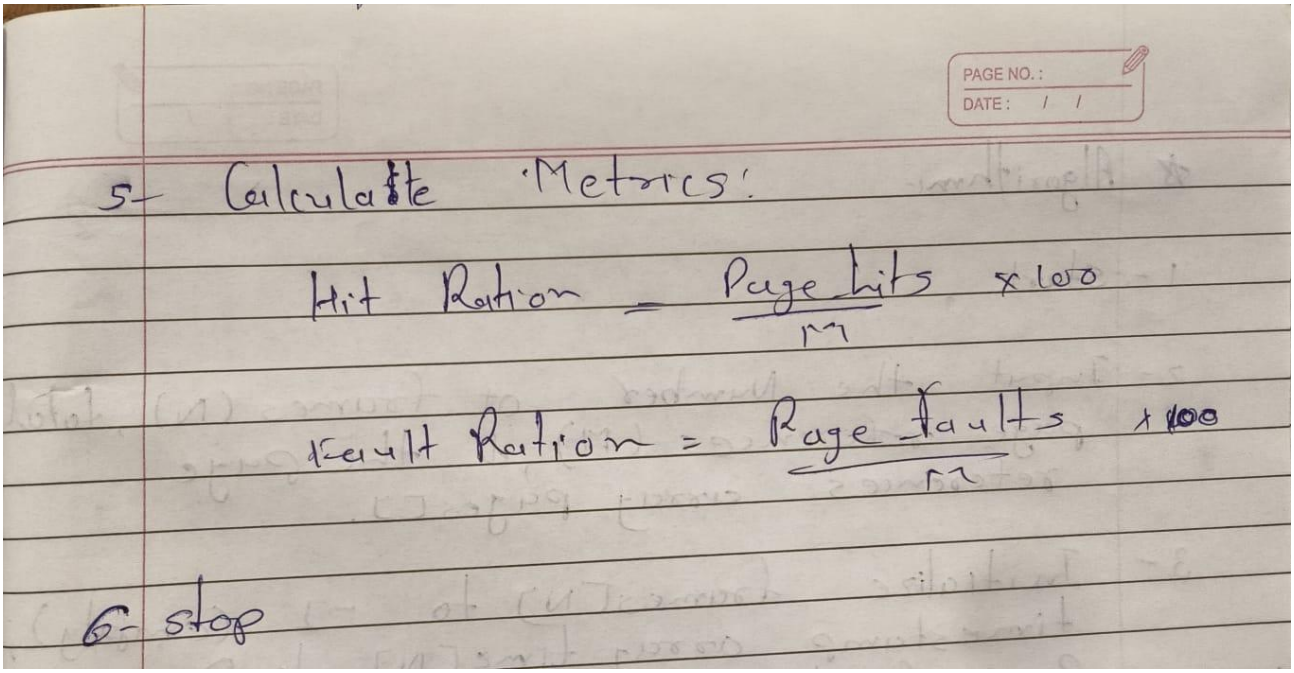
2- Input the Number of frames (N), total page references (M), & the page references array pages[].

3- Initialize frames[N] to -1 (empty), timestamp array time[N] to 0, Page_faults = 0, & Page_hits = 0.

4-

4- for each page P in pages[] (using a counter variable t for access time):

- checks for Hit: if P exists in frames[i]:
  - update time[i] = t
  - Increment page_hits
- If Page Fault (P not in frames[]):
  - If empty frame exists (frames[i] == -1): Insert P into frames[i] & set time[i] = t.
  - If frames are full: Find index k with minimum value in time[], replace frames[k] with P, & update time[k] = t.
  - Increment page_faults.



Program:

Code 1 :-

```
#include <stdio.h>

#define FRAMES 4

int findLRU(int time[], int n) {
   int min = time[0], pos = 0; for
   (int i = 1; i < n; ++i) {
      if (time[i] < min) {
         min = time[i]; pos
         = i;
      }
   }
   return pos;
}

int main() {
   int ref_str[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1};
   int n = sizeof(ref_str) / sizeof(ref_str[0]);

   int frames[FRAMES]; int
   time[FRAMES];
   int page_faults = 0, hits = 0, counter = 0;

   for (int i = 0; i < FRAMES; ++i) {
      frames[i] = -1;
      time[i] = 0;
   }
```

```
printf("Ref\tFrame 1\tFrame 2\tFrame 3\tFrame 4\tStatus\n");
printf(" ------------------------------------------------------ \n");

for (int i = 0; i < n; ++i) {
   int page = ref_str[i]; int
   flag1 = 0, flag2 = 0;

   for (int j = 0; j < FRAMES; ++j) { if
      (frames[j] == page) {
         counter++; time[j]
         = counter; hits++;
         flag1 = flag2 = 1;
         break;
      }
   }

   if (flag1 == 0) {
      for (int j = 0; j < FRAMES; ++j) { if
         (frames[j] == -1) {
            counter++;
            page_faults++;
            frames[j] = page;
            time[j] = counter;
            flag2 = 1;
            break;
         }
      }
   }

   if (flag2 == 0) {
      int pos = findLRU(time, FRAMES); counter++;
      page_faults++;
      frames[pos] = page;
      time[pos] = counter;
   }

   printf("%d\t", page);
   for (int j = 0; j < FRAMES; ++j) { if
      (frames[j] != -1)
         printf("%d\t", frames[j]); else
         printf("-\t");
   }
   if (flag1 == 1)
      printf("Hit\n");
   else
      printf("Fault\n");
}
```

```
    printf(" ---------------------------------------------------- \n");
    printf("Total Page Faults: %d\n", page_faults);
    printf("Total Hits: %d\n", hits);
    printf("Hit Ratio: %.2f%%\n", ((float)hits / n) * 100); printf("Fault
    Ratio: %.2f%%\n", ((float)page_faults / n) * 100);

    return 0;
}
```

Output:

```
vatsal@ubuntu-VirtualBox:~$ nano lru1.c
vatsal@ubuntu-VirtualBox:~$ gcc lru1.c -o lru1
vatsal@ubuntu-VirtualBox:~$ ./lru1
Ref     Frame 1 Frame 2 Frame 3 Frame  4 Status
-----------------------------------------------------
7       7       -       -       -       Fault
0       7       0       -       -       Fault
1       7       0       1       -       Fault
2       7       0       1       2       Fault
0       7       0       1       2       Hit
3       3       0       1       2       Fault
0       3       0       1       2       Hit
4       3       0       4       2       Fault
2       3       0       4       2       Hit
3       3       0       4       2       Hit
0       3       0       4       2       Hit
3       3       0       4       2       Hit
2       3       0       4       2       Hit
1       3       0       1       2       Fault
-----------------------------------------------------
Total Page Faults: 7
Total Hits: 7
Hit Ratio: 50.00%
Fault Ratio: 50.00%
```

Code 2 :-

```
#include <stdio.h>

#define FRAMES 4

int findLRU(int time[], int n) {
    int min = time[0], pos = 0; for
    (int i = 1; i < n; ++i) {
        if (time[i] < min) {
            min = time[i]; pos
            = i;
        }
    }
    return pos;
```

```
}

int main() {
    int ref_str[] = {1, 2, 3, 4, 2, 5, 1, 2, 5, 6, 2, 4, 3};
    int n = sizeof(ref_str) / sizeof(ref_str[0]);

    int frames[FRAMES]; int
    time[FRAMES];
    int page_faults = 0, hits = 0, counter = 0;

    for (int i = 0; i < FRAMES; ++i) {
        frames[i] = -1;
        time[i] = 0;
    }

    printf("Ref\tFrame 1\tFrame 2\tFrame 3\tFrame 4\tStatus\n");
    printf(" ---------------------------------------------------- \n");

    for (int i = 0; i < n; ++i) {
        int page = ref_str[i]; int
        flag1 = 0, flag2 = 0;

        for (int j = 0; j < FRAMES; ++j) { if
            (frames[j] == page) {
                counter++; time[j]
                = counter; hits++;


                flag1 = flag2 = 1;
                break;
            }
        }

        if (flag1 == 0) {
            for (int j = 0; j < FRAMES; ++j) { if
                (frames[j] == -1) {
                    counter++;
                    page_faults++;
                    frames[j] = page;
                    time[j] = counter;
                    flag2 = 1;
                    break;
                }
            }
        }

        if (flag2 == 0) {
            int pos = findLRU(time, FRAMES); counter++;
            page_faults++;
            frames[pos] = page;
            time[pos] = counter;
        }

        printf("%d\t", page);
```

```
        for (int j = 0; j < FRAMES; ++j) { if
            (frames[j] != -1)
                printf("%d\t", frames[j]); else
                printf("-\t");
        }
        if (flag1 == 1)
            printf("Hit\n");
        else
            printf("Fault\n");
    }

    printf(" ---------------------------------------------------- \n");
    printf("Total Page Faults: %d\n", page_faults);
    printf("Total Hits: %d\n", hits);
    printf("Hit Ratio: %.2f%%\n", ((float)hits / n) * 100); printf("Fault
    Ratio: %.2f%%\n", ((float)page_faults / n) * 100);

    return 0;
}
```

Output :

```
vatsal@ubuntu-VirtualBox:~$ nano lru2.c
vatsal@ubuntu-VirtualBox:~$ gcc lru2.c -o lru2
vatsal@ubuntu-VirtualBox:~$ ./lru2
Ref     Frame 1 Frame 2 Frame 3 Frame  4 Status
-----------------------------------------------------
1       1       -       -       -       Fault
2       1       2       -       -       Fault
3       1       2       3       -       Fault
4       1       2       3       4       Fault
2       1       2       3       4       Hit
5       5       2       3       4       Fault
1       5       2       1       4       Fault
2       5       2       1       4       Hit
5       5       2       1       4       Hit
6       5       2       1       6       Fault
2       5       2       1       6       Hit
4       5       2       4       6       Fault
3       3       2       4       6       Fault
-----------------------------------------------------
Total Page Faults: 9
Total Hits: 4
Hit Ratio: 30.77%
Fault Ratio: 69.23%
```

Conclusion:

Through this experiment, I learned how the Least Recently Used (LRU) page replacement algorithm minimizes page faults by evicting the page that has remained unaccessed for the longest duration. I gained practical knowledge of implementing LRU in C using timestamp tracking to monitor memory frame activity effectively. Analyzing sample reference strings across four page frames helped me understand the direct impact of frame size on hit and fault ratios. Ultimately, this experiment deepened my understanding of dynamic virtual memory management and the trade-offs between algorithm efficiency and system overhead in operating systems.

Assessment Schemes:

Attendance	Discipline	Short oral	Correctness of Lab Report	Timely completion of Lab Report		Total marks (10)	Signature of Teacher with Date